

Learning to Transfer for Evolutionary Multitasking

Sheng-Hao Wu[✉], *Member, IEEE*, Yuxiao Huang, Xingyu Wu[✉], *Member, IEEE*,
Liang Feng[✉], *Senior Member, IEEE*, Zhi-Hui Zhan[✉], *Fellow, IEEE*, and Kay Chen Tan[✉], *Fellow, IEEE*

Abstract—Evolutionary multitasking (EMT) is an emerging approach for solving multitask optimization problems (MTOPs) and has garnered considerable research interest. The implicit EMT is a significant research branch that utilizes evolution operators to enable knowledge transfer (KT) between tasks. However, current approaches in implicit EMT face challenges in adaptability, due to the limited use of different evolution operators with different parameter settings and insufficient utilization of evolutionary states for performing KT. This results in suboptimal exploitation of implicit KT's potential to tackle a variety of MTOPs. To overcome these limitations, we propose a novel learning-to-transfer (L2T) framework to automatically discover efficient KT policies for the MTOPs at hand. Our framework conceptualizes the KT process as a learning agent's sequence of strategic decisions within the EMT process. We propose an action formulation for deciding when and how to transfer, a state representation with informative features of evolution states, a reward formulation concerning convergence and transfer efficiency gain, and the environment for the agent to interact with MTOPs. We employ an actor-critic network structure for the agent and learn the policy via proximal policy optimization. This learned agent can be integrated with various evolutionary algorithms, enhancing their ability to address unseen MTOPs. Comprehensive empirical studies on both synthetic and real-world MTOPs, encompassing diverse intertask relationships, function classes, and task distributions are conducted to validate the proposed L2T framework. The results show a marked improvement in the adaptability and performance of implicit EMT when solving a wide spectrum of unseen MTOPs.

Index Terms—Evolutionary multitasking (EMT), learning to optimize, reinforcement learning (RL).

Received 3 February 2025; revised 7 April 2025; accepted 9 April 2025. Date of publication 29 April 2025; date of current version 21 June 2025. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant U21A20512, and Grant 62176094; in part by the Fundamental Research Funds for the Central Universities, Nankai University under Grant 078-63253247; and in part by the Research Grants Council of the Hong Kong SAR, under Grant C5052-23G, Grant PolyU 15229824, Grant PolyU 15218622, and Grant PolyU 15215623. This article was recommended by Associate Editor Y. Shi. (*Corresponding author: Kay Chen Tan.*)

Sheng-Hao Wu, Yuxiao Huang, Xingyu Wu, and Kay Chen Tan are with the Department of Data Science and Artificial Intelligence, The Hong Kong Polytechnic University, Hong Kong SAR, China (e-mail: kctan@polyu.edu.hk).

Liang Feng is with the College of Computer Science, Chongqing University, Chongqing 400044, China, and also with the Chongqing Key Laboratory of Big Data Intelligence and Privacy Computing, Chongqing 400044, China (e-mail: liangf@cqu.edu.cn).

Zhi-Hui Zhan is with the College of Artificial Intelligence, Nankai University, Tianjin 300350, China (e-mail: zhanapollo@163.com).

This article has supplementary material provided by the authors and color versions of one or more figures available at <https://doi.org/10.1109/TCYB.2025.3561518>.

Digital Object Identifier 10.1109/TCYB.2025.3561518

I. INTRODUCTION

IN THE real world, complex optimization problems abound. Traditional numerical optimization algorithms often struggle to solve these problems efficiently due to their nonconvexity and black-box properties. Against this backdrop, evolutionary computation (EC) [1], [2] algorithms have been widely adopted to tackle such challenging optimization problems. However, these algorithms are typically designed to solve one task at a time, which can be computationally inefficient when dealing with a multitude of tasks that may share some similarities and commonalities. Recognizing this limitation, an advanced search paradigm, known as evolutionary multitasking (EMT) [3], [4], [5], has recently emerged. Unlike traditional EC algorithms, EMT does not handle each task in isolation. Instead, it seeks to solve multiple optimization tasks simultaneously by leveraging the inherent complementarity and sharing useful information between them to improve search efficiency. Since the emergence of the EMT paradigm, numerous technical advancements have been achieved across methodological [6], [7], theoretical [8], [9], and application [10], [11] dimensions. The problem of solving multiple tasks concurrently is called the multitask optimization problem (MTOP) while the technique of sharing useful information is referred to as knowledge transfer (KT) [12]. As the core of the EMT, the KT process plays a vital role in achieving satisfactory optimization performance.

Existing EMT algorithms can be classified into two categories according to the design of KT: 1) explicit [13] and 2) implicit EMT [3]. The explicit EMT algorithms are characterized by involving an online learning process and a transformation process for extracting the cross-task knowledge. This online learning process usually uses transfer learning algorithms with a defined objective for measuring knowledge utility. Despite having advantage in bridging the gap between heterogeneous tasks, the explicit EMT inevitably incurs extra computational cost. In contrast, the implicit EMT is efficient and straightforward because it typically employs evolution operators on the solutions of multiple tasks to directly transfer knowledge with a small computational overhead. Therefore, implicit EMT has been successfully applied to solve various complex optimization problems [14], [15].

Despite its promise, a proper design of the KT process in implicit EMT requires considering the issues of *when to transfer* and *how to transfer* to ensure transfer effectiveness. The “when to transfer” issue refers to dynamically adapting the implicit KT intensity while the “how to transfer” issue refers to the process of extracting knowledge by evolution operators, such as crossover [3], from the source task. However, existing

designs of KT intensity adaptation rely on limited information about the current evolution state such as the KT feedback [16] and estimated intertask similarity [17] based on manually defined heuristic rules. This insufficient use of evolution information can lead to suboptimal designs of KT. Moreover, most KT processes in implicit EMT rely on a single evolution operator with fixed parameters [3], a few evolution operators associated with fixed parameter settings [18], or single evolution operator with a few parameter settings [12] to perform KT. In particular, the previous work [3] uses a simulated binary crossover (SBX) with fixed parameter, [18] employs six different crossover operators, and [12] includes a single mutation operator with three different parameter settings. The optimization tasks in practical MTOPs often display a wide array of unique characteristics, necessitating a diverse range of evolution operators for effective adaptation. This constrained use of evolution operators could hinder the ability of implicit KT to adapt to new and unseen MTOPs with varied task distributions. Therefore, the challenge lies in how to better adapt the KT intensity and the evolution operator within the KT process to suit the specific needs of MTOPs that have tasks originating from a particular distribution. Additionally, designing a suitable KT process is usually driven by human expertise and requires considerable trial-and-error efforts on the problem set of interest. This human-dependent design process hinders users from applying EMT to solve a wide range of real-world MTOPs.

Given the above challenges discussed, it is apparent that a more adaptive and automatic KT process is needed. To this end, we take cues from the *learning to optimize* [19] in the meta-learning domain to address these challenges. Learning to optimize is a technique that aims to automatically discover how to perform optimization based on the experience gained from solving a set of problems using machine learning methods such as reinforcement learning (RL). In specific, we propose a novel learning-to-transfer (L2T) framework in this article to address these challenges. First, we observe that the KT process within the implicit EMT process can be formulated as a Markov decision process (MDP) to maximize the transfer quality and overall search performance. Based on this formulation, the KT process can be regarded as the sequential decisions on when and how to transfer made by an agent associated with certain parameters within the implicit EMT process. By parameterizing the agent as an actor-critic network, we propose to learn well-performing implicit KT policy by proximal policy optimization (PPO) [20]. Specifically, PPO serves to train the network parameters so that the agent can maximize the transfer quality and search efficiency. Finally, the learned agent can be seamlessly incorporated into different EC algorithms to solve unseen MTOP instances. To our knowledge, little research has been devoted to improve the adaptability and automation of the KT process by jointly addressing when to transfer and how to transfer using RL techniques.

The contributions of this article are twofold. First, we introduce an L2T framework that refines the adaptability of the KT process within the implicit EMT process. By learning from optimization experiences, this framework facilitates the implicit EMT effectively address a broader spectrum

of MTOPs with diverse task similarities and relationships, compared to existing EMT algorithms. Second, to evaluate the adaptability of the EMT algorithms, we propose a test suite with diverse task distributions and function characteristics and conduct comprehensive experiments to validate the effectiveness of the proposed L2T framework on this test suite.

The remainder of this article is organized as follows. Section II provides an overview of EMT, related works on KT for EMT, and learning to optimize. Section III introduces the proposed L2T framework and its implementation in detail. Section IV presents the experimental studies to evaluate the performance and verify the effectiveness of the proposed L2T framework. Finally, the concluding remarks and future work are given in Section V.

II. PRELIMINARIES

A. Evolutionary Multitasking

The EMT is a paradigm for optimizing multiple tasks using EC algorithms—such as differential evolution (DE) [21]—as the base solver, incorporated with a KT process. We briefly review the evolution process by the base solver DE and the probabilistic sampling model of the KT-based offspring generation in EMT, as they form the cornerstone of our proposed method. Let K denote the number of tasks to be optimized in an MTOP instance. At generation g , to produce the i th offspring $u_{k,i}$ by the DE algorithm for the k th optimization task f_k ($k = 1, \dots, K$), two steps—mutation and crossover—are performed. Although many mutation and crossover operators have been proposed in DE, we employ the widely used “DE/rand/1” mutation and binomial crossover for brevity. The mutation operator first generates a trial vector $v_{k,i}$ as

$$v_{k,i} = \underbrace{X_{k,r_1}}_{\text{base vector}} + F \cdot \underbrace{(X_{k,r_2} - X_{k,r_3})}_{\text{differential vector}} \quad (1)$$

where X_k is the parent population at generation g , F is the scaling factor in DE, and r_1, r_2, r_3 are distinct indices randomly selected from $\{1, \dots, N\}$, where N denotes the population size. In this mutation operation, X_{k,r_1} is called the *base vector* and $(X_{k,r_2} - X_{k,r_3})$ is called the *differential vector*. Then, an offspring solution $u_{k,i}$ is generated as

$$u_{k,i,d} = \begin{cases} v_{k,i,d} & \text{if } \text{rand}_d(0, 1) \leq CR \\ & \text{or } d = \text{randint}_i(1, D) \\ X_{k,i,d} & \text{otherwise} \end{cases} \quad (2)$$

where d is the dimension index, D denotes the task dimension, and CR is a parameter to control crossover rate. If the offspring is generated solely by the DE operators (1)–(2), the process is equivalent to single-task optimization because no additional information from other tasks is incorporated. Hence, the KT operation is introduced in EMT by combining information from other tasks. According to [22], the KT process for generating an offspring x_k for the target task f_k using the base solver $\mathcal{A}$ (e.g., DE) is modeled as follows:

$$p(x_k | \{\mathcal{M}_j\}_{j=1}^K, \mathcal{A}) = \alpha_k p(x_k | \mathcal{M}_k, \mathcal{A}) + \sum_{j \neq k} \alpha_j p(x_k | \mathcal{M}_j, \mathcal{A}) \quad (3)$$

where $\mathcal{M}$ denotes the experience collected along the search process until the current generation g . In the case of DE, $\mathcal{M}$ is represented as the current population X_k and its fitness Y_k , while $\mathcal{A}$ is represented as the mutation and crossover operator. For simplicity, we assume all tasks use the same base solver $\mathcal{A}$. Moreover, α_k is a parameter to balance between self-evolution (i.e., using the base solver with target task-only information) and KT (i.e., combining the information from source tasks) to generate offspring. Different KT designs for controlling α_k range from fixed [3] to adaptive [23].

A limitation of this model is that it only considers the mixture of the transferred solutions that are directly sampled from the source task distribution. However, as revealed in the previous study [24], performing domain adaptation to transform the source task distribution can enhance the KT utility. Hence, we can rewrite the model to a more general form to cover the transformation-based KT process as

$$p(x_k|\{\mathcal{M}_j\}_{j=1}^K, \mathcal{A}, \mathcal{K}) = \alpha_k p(x_k|\mathcal{M}_k, \mathcal{A}) + (1 - \alpha_k) p(x_k|\{\mathcal{M}_j\}_{j=1}^K, \mathcal{A}, \mathcal{K}). \quad (4)$$

Herein, α_k is a parameter that controls *when to transfer* for a task f_k . $\mathcal{K}$ is typically a function that takes the information $\{X_j\}_{j=1}^K$ of all the tasks as input and undergoes a knowledge extraction to produce a transferred offspring $v_{k,i}$, i.e., $\mathcal{K}:\{X_j\}_{j=1}^K \rightarrow v_{k,i}$. Different designs of KT operators can be regarded as an implementation of $\mathcal{K}$. Hence, $\mathcal{K}$ is a function concerning *how to transfer*. Here, we define an algorithm as an *explicit* EMT if it incorporates both an online learning process and a solution transformation process for further KT; otherwise, it is categorized as *implicit*. For example, the multifactorial evolutionary algorithm (MFEA) [3] belongs to the implicit EMT algorithms because the KT is achieved by the evolution operator without any transformation. Moreover, we provide a literature review of existing KT methods addressing the issues of when to transfer and how to transfer, as well as a discussion on the similarities and differences between the L2T framework and existing EMT methods. Due to page limits, this review is presented in Section S.I of the supplementary material [25].

Despite the rapid development of various KT designs, the issue of the limited adaptability of the KT process remains an open research problem. The adaptability of the KT process is defined as the average optimization performance on a set of unseen MTOPs that have not been seen during the development and tuning process of the EMT algorithm. We conduct experiments to examine the adaptability of existing KT processes [21], [26], [27], [28], [29], each focusing on when and how to transfer (see Section IV-B). We find that the current implicit EMT algorithms exhibit a restricted capacity for adapting to different MTOPs with diverse and complex task distributions, warranting further study. Moreover, there is a need for a more automatic KT process that can learn effective policies for deciding when and how to transfer based on the specific MTOPs at hand.

B. Learning to Optimize

Learning to optimize [30] is an active research branch that uses machine learning techniques to improve optimization

efficiency. Among various machine learning techniques, RL is popular for guiding the search process at each step. The evolutionary process can be viewed as a sequential decision-making problem, involving multiple sampling and distribution update steps to approach the optima. Many researchers have realized the potential of RL in assisting the EC to tackle this decision-making problem and proposed some RL methods to automatically design EC structure [31] and control EC parameters [32]. Additionally, several studies have attempted to adopt artificial neural networks to assimilate and leverage evolutionary knowledge by predicting the evolutionary component [33] and evolutionary direction [34] throughout the search process, yielding promising outcomes. Note that as a component of EMT, the goal of the KT process is to make the proper decisions on when and how to perform transfer between tasks to achieve the best optimization performance over all the tasks. Intuitively, this goal shares certain similarities with existing works on using RL to automate the evolutionary process [35]. Similar works [36] using RL to assist EMT have been studied. However, the adaptation component in this work is only limited to the *when to transfer* issue, which cannot fully unlock the potential of RL. Moreover, many existing works employing RL in the domain of learning to optimize can be viewed as instances of policy transfer [37], i.e., training the agent on diverse function instances and testing it on new function instances sharing similar landscape characteristics. Our work remarkably differs from these approaches in that we are the first, to the best of our knowledge, to explore RL for transferring knowledge between tasks by jointly considering *when to transfer* and *how to transfer* under the EMT paradigm. Concretely, the contribution is to model the KT between tasks along the EMT process as MDP and solving it with RL.

III. PROPOSED METHOD

A. L2T Workflow

The diagram of the proposed L2T framework is shown in Fig. 1. Generally, L2T contains two stages, namely, the learning and the inference stage. The learning stage receives as input a set of task instances comprising diverse optimization tasks, which are used to sample and construct MTOP instances. The learning stage comprises an outer loop for learning the KT policy and an inner loop, referred to as a *rollout*, during which the agent interacts with the environment. In the learning stage, the EMT environment continuously generates new MTOP instances for an agent to solve. In the rollout process, the agent undergoes an EMT process by taking actions to produce offspring. Specifically, for each time step g (i.e., a generation in the EC process), the agent receives a state after performing feature extraction on the populations X_{g-1} and fitness Y_{g-1} . Then, the agent outputs an action to perform KT-based sampling to obtain offspring population X_g . A complete rollout process is called an *episode*. In an episode, the state, action, and reward of each time step will be collected as rollout data. After a certain number of episodes have been completed, the agent updates the parameter θ according to the RL algorithm. When the learning is finished, the agent with optimized parameters θ^* can be directly plugged and used in EMT algorithms to solve new and unseen MTOPs. The major

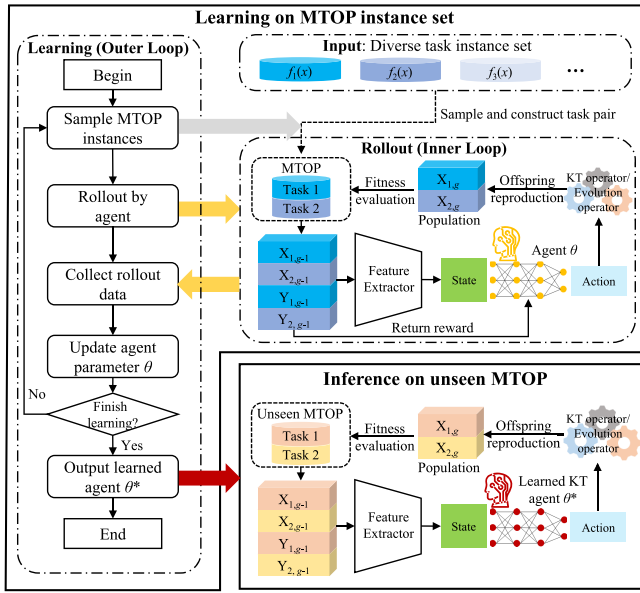


Fig. 1. Diagram of the proposed L2T framework.

components of the L2T framework include: 1) the definition of the RL components (Section III-B); 2) the procedures for learning the agent; and 3) the method for integrating the agent into the EMT algorithm during the inference stage (Section III-C). It should be noted that the main computational cost occurs in the learning stage, while the time complexity of L2T during the inference stage is low (see Section III-D for analysis).

B. L2T Component Design

1) *MDP Formulation of KT Process*: We first formulate the KT process within the implicit EMT as an MDP problem that takes high-valued actions (i.e., deciding when and how to transfer) to maximize the cumulative reward (i.e., yielding the best results when the optimization finishes). Normally, an MDP can be represented as a tuple (a, s, r, π, p_{tr}) where a denotes the action, s denotes the state, r denotes the reward function, π denotes the probability distribution of the agent's action parameterized by θ , and p_{tr} denotes the state transition probability. The objective is to find an optimal agent $\pi^*(a|s; \theta)$ parameterized by θ to realize $\max_{\pi} \mathbb{E}_{\tau} [p(\tau; \pi) R(\tau)]$, where $\tau = (s_0, a_0, r_0, \dots, s_T, a_T, r_T)$ denotes the realization of a trajectory, T denotes the time horizon (i.e., maximum generations in EMT), and $R(\tau) = \sum_{t=0}^T r_t$ denotes the cumulative reward. The probability of a trajectory τ is affected by π and p_{tr} , i.e., $p(\tau; \pi) = \prod_{t=0}^T \pi(a_t|s_t; \theta) p_{tr}(s_{t+1}|s_t, a_t)$. The formulations of action a , state s , reward r , agent structure $\pi(a|s; \theta)$, and the environment dynamics p_{tr} will be introduced in the following.

2) *Action Formulation*: We first briefly revisit some KT operators that are related to our design of the action space for the KT process. Some representative KT operators include linear combination-based crossover [18], base vector transfer [28], and differential vector transfer [27]. The linear combination-based crossover, which includes arithmetic crossover [18] and SBX [3], performs a convex combination

TABLE I
INTERPRETATION OF DIFFERENT KT ACTIONS

Action parameters			Translated operation	
$a_{k,1}$	$a_{k,2}$	$a_{k,3}$	When to transfer	How to transfer
0	/	/	Without KT	Self evolution by $\mathcal{A}$ (e.g., Eq. (1))
0	1	KT probability=0.5	Transfer differential vector	(Eq. (7))
0.5	0	KT probability=0.5	Crossover on source and target solutions (Eq. (5)) plus differential vector	
1	0	KT probability=0.5	Transfer source solutions as base vector	(Eq. (6))
1	1	KT probability=0.5	Transfer sampled solution based on source population	(Eq. (8))

of target and source solutions to generate offspring $v_{k,i}$ as follows:

$$v_{k,i} = \lambda \cdot X_{k,r_1} + (1 - \lambda) \cdot X_{j,r_2} \quad (5)$$

where $j \neq k$ is a randomly selected source task index from $\{1, \dots, K\}$ and $\lambda \in [0, 1]$ is a random variable that is drawn from a predefined distribution. The base vector transfer is

$$v_{k,i} = X_{j,r_1} + F \cdot (X_{k,r_2} - X_{k,r_3}) \quad (6)$$

while the differential vector transfer is

$$v_{k,i} = X_{k,r_1} + F \cdot (X_{j,r_2} - X_{j,r_3}). \quad (7)$$

Moreover, directly transferring the solution sampled from the source population using an evolutionary operator—similar to the approach in [29] and [38]—is also a well-known KT operator, given by

$$v_{k,i} = X_{j,r_1} + F \cdot (X_{j,r_2} - X_{j,r_3}) \quad (8)$$

with base vector X_{j,r_1} different from X_{k,r_1} in (7). It is widely studied that different KT operators have different transfer biases and favor different kinds of task similarity and evolution states [24]. This inspires us to make use of the complementarity of different KT operators and devise a more generalized KT action representation that can adapt to diverse evolution states and task similarities. Hence, combining the KT probabilistic model in (4) and existing KT operators, we derive the sampling model for an offspring $v_{k,i}$ at generation g as

$$v_{k,i} \sim (1 - 0.5 \cdot a_{k,1}) \cdot p(v_k|X_k, \mathcal{A}) + 0.5 \cdot a_{k,1} \cdot p(v_k|\{X_j\}_{j=1}^K, \mathcal{K}) \quad (9)$$

where $v_{k,i} \sim p(v_k|\{X_j\}_{j=1}^K, \mathcal{K})$ is defined as

$$v_{k,i} = (1 - a_{k,2}) \cdot X_{k,r_1} + a_{k,2} \cdot X_{j,r_2} + F \cdot (1 - a_{k,3}) \cdot (X_{k,r_3} - X_{k,r_4}) + F \cdot a_{k,3} \cdot (X_{j,r_5} - X_{j,r_6}) \quad (10)$$

where $r_1, r_2, r_3, r_4, r_5, r_6$ are randomly drawn from $\{1, \dots, N\}$. $a_{k,1}, a_{k,2}, a_{k,3} \in [0, 1]$ are three continuous action parameters output by an agent for task f_k and can be varied among generations. After generating a trial vector $v_{k,i}$, the binomial crossover operation is put forward to produce the final offspring $u_{k,i}$. According to the analysis in [23], we

TABLE II
STATE FEATURES FOR THE INPUT OF THE LEARNABLE AGENT

Feature type	Notation and definition	Range	Meaning
Common feature	$O_{c,1} = g/G_{\max}$	[0, 1]	Ratio of current generation g
	$O_{c,2} = d(x_1^*, x_2^*)/\sqrt{D}$	[0, 1]	Distance between best-found solutions till current generation g
	$O_{c,3} = d(\mu_1, \mu_2)/\sqrt{D}$	[0, 1]	Distance between first-moment statistics of current population
	$O_{c,4} = d(\sigma_1, \sigma_2)/\sqrt{0.5D}$	[0, 1]	Distance between second-moment statistics of current population
Task-specific feature	$O_{t,k,1} = n_{k,\text{stag}}/G_{\max}$	[0, 1]	Ratio of stagnating generations
	$O_{t,k,2} = \text{flag}_{k,\text{improved}}$	[0, 1]	Whether best solution is improved in previous generation
	$O_{t,k,3} = q_{k,\text{KT}}$ (Eq. (11))	[0, 1]	Transfer quality of the previous knowledge transfer action
	$O_{t,k,4} = \text{mean}(\sigma_k)$	[0, 1]	Average deviation of the current population over all dimensions
	$O_{t,k,5} = a_{k,1,g-1}$	[0, 1]	Previous KT action taken for deciding when to transfer
	$O_{t,k,6} = a_{k,2,g-1}$	[0, 1]	Previous KT action taken for deciding how to transfer
	$O_{t,k,7} = a_{k,3,g-1}$	[0, 1]	Previous KT action taken for deciding how to transfer

impose the restriction on the probability of self-evolution by base solver $\mathcal{A}$ to be larger than 0.5 under truncated selection mechanism, i.e., $\alpha_k = 1 - 0.5 \cdot a_{k,1} \geq 0.5$, to guarantee convergence. The main feature of this action formulation is that by setting different $a_{k,1}, a_{k,2}, a_{k,3}$, the mutation operator in (1) and existing KT operators in (5)–(8) can be recovered from (10). That is, the above-mentioned self-evolution operator in (1) and KT operators in (5)–(8) are a subset of the proposed action space. The detailed interpretation of different KT actions is shown in Table I. Note that since the action parameters $a_{k,1}, a_{k,2}$, and $a_{k,3}$ are continuous within [0, 1], choosing values in the interval (0, 1) corresponds to applying a combination of multiple KT operators. In this way, we have expanded the number of possible KT actions from a few (5)–(8) to infinitely many. Intuitively, we expect that more candidate actions of linearly combining KT operators can provide more diverse behaviors and potentially improve the capacity of the KT process to adapt to more diverse MTOPs.

3) *State Representation*: Now, we proceed to define what information should be fed into the agent for supporting it to make high-quality decisions. A naive way is to directly feed all the populations and fitness of tasks at generation g to the agent without any processing. However, this input yields a high dimensionality, i.e., $K \cdot N \cdot (D+1)$, where K denotes the number of tasks and $D+1$ represents the combined dimensions of the solution and its fitness. Since not all information in the population is useful, the large input may distract the agent from attending to useful information for decision-making. Therefore, we extract features from the populations to facilitate a more efficient learning process. Existing adaptive KT designs prefer using feedback of KT [26] and estimated intertask similarities [17] as features for adapting intensity parameters. By combining these two ideas and adding more informative features, the proposed state features are given in Table II. The incorporated features can be divided into common features O_c and task-specific features O_t . The common features are shared among all the tasks while the task-specific features are calculated for each task independently. The common features include: 1) the timestamps of the current generation $O_{c,1} = g/G_{\max}$, where g is the current generation and $G_{\max}$ is the maximum generations and 2) three distance metrics $O_{c,2}, O_{c,3}, O_{c,4}$ to estimate task similarity, where $d(\cdot, \cdot)$ is the Euclidean distance. Herein, x_k^* denotes the best-found

solution on task k , while μ_k and σ_k denote the mean and standard deviation vector (D -dimensional) calculated on every dimension of the population X_k , respectively. The task-specific features, with respect to each task k , include the stagnation state $O_{t,k,1} = n_{k,\text{stag}}/G_{\max}$, where $n_{k,\text{stag}}$ is the number of stagnating generations, the improvement state $O_{t,k,2} = \text{flag}_{k,\text{improved}} \in \{0, 1\}$ indicating whether the best solution has been improved in previous generation, the transfer quality of previous KT action $O_{t,k,3} = q_{k,\text{KT}}$, the average deviation of the current population $O_{t,k,4} = \text{mean}(\sigma_k)$ over all dimensions of the vector σ_k , and the previous KT action $O_{t,k,5}, O_{t,k,6}, O_{t,k,7}$. In particular, σ_k is defined as the vector containing standard deviations for each dimension of population k . Specifically, given the fitness of the population at the g th generation $Y_{k,g} = \{y_{k,1}, \dots, y_{k,N}\}$ and the offspring produced by KT, denoted as $Y_{k,\mathcal{K}} = \{y_{k,\mathcal{K},1}, \dots, y_{k,\mathcal{K},N_{k,\text{KT}}}\}$, where $N_{k,\text{KT}}$ denotes the number of KT-generated solutions for the task f_k , the transfer quality is calculated as

$$q_{k,\text{KT}} = \begin{cases} \frac{\sum_{i=1}^{N_{k,\text{KT}}} |\{y_{k,g} \in Y_{k,g} : y_{k,\mathcal{K},i} < y_{k,g}\}|}{N_{k,\text{KT}} \cdot N}, & \text{if } N_{k,\text{KT}} > 0 \\ 0, & \text{otherwise} \end{cases} \quad (11)$$

where $|\cdot|$ is the cardinality of a set. To make the magnitude of different features compatible, we normalize them to the range [0, 1]. Hence, given K tasks, the state dimension after the feature extraction is $|O_c| + |O_t| = 4 + 7 \cdot K$.

4) *Reward Design*: The reward serves as a temporary signal for incentivizing the agent to learn good policies. The objective for the agent in EMT is to achieve the targeted optimization accuracy ξ at generation g , i.e., $\mathbb{I}(f_k(x_{k,g}^*) - f_k^* < \xi)$, where $x_{k,g}^*$ is the best-found solution by the EMT algorithm until generation g , $\mathbb{I}(\cdot)$ is the indicator function, and $f_k^* = \min_x f_k(x)$ is the optimal objective value. Simply giving the reward to the agent at the end of a rollout process if the target accuracy is reached will lead to the sparse reward issue [39], which is difficult for the agent to learn. To alleviate this issue, we aggregate two additional reward terms, namely convergence gain $r_{k,\text{conv}}$ and KT gain $r_{k,\text{KT}}$. The reward over all tasks is defined as

$$r = \sum_{k=1}^K r_k \quad (12)$$

where r_k defines the subreward on task f_k as

$$r_k = b_1 \cdot r_{k,\text{conv}} + b_2 \cdot r_{k,\text{KT}} + b_3 \cdot \mathbb{I}(f_k(x_{k,g}^*) - f_k^* < \xi) \quad (13)$$

where

$$r_{k,\text{conv}} = -(f_k(x_{k,g}^*) - f_k^*) / (f_k(x_{k,1}^*) - f_k^*) \quad (14)$$

$$r_{k,\text{KT}} = \mathbb{E}_{x_k \sim p(x|X_k, \mathcal{A})} [f_k(x_k)] - \mathbb{E}_{x_k \sim p(x|X_k, \mathcal{K})} [f_k(x_k)] \quad (15)$$

and b_1, b_2, b_3 are hyperparameters. One important property of $r_{k,\text{KT}}$ is that it rewards the agent with a positive signal for producing a higher expectation of fitness quality by KT than the base solver $\mathcal{A}$ and a negative reward otherwise. This reward design is expected to drive the agent to balance between the risks of negative transfer and the performance gain brought by KT. In practice and for the sake of computational simplicity, $r_{k,\text{KT}}$ is approximated as the difference in the average normalized scores of KT-generated offspring solutions and that of base solver-generated offspring solutions, i.e.,

$$r_{k,\text{KT}} \approx \frac{1}{N_{\mathcal{K}}} \sum_{j=1}^{N_{\mathcal{K}}} s(y_{k,\mathcal{K},j}) - \frac{1}{N_{\mathcal{A}}} \sum_{j=1}^{N_{\mathcal{A}}} s(y_{k,\mathcal{A},j}). \quad (16)$$

$y_{k,\mathcal{K},j}$ denotes the fitness of j th offspring produced by $\mathcal{K}$ on task f_k and $y_{k,\mathcal{A},j}$ denotes the fitness of j th offspring produced by $\mathcal{A}$. $s(y)$ denotes the fitness score of an offspring at generation g , and is calculated as a normalized value within $[0, 1]$ over the parent population of the corresponding task, i.e.,

$$s(y) = |\{y_{k,g} \in Y_{k,g} : y < y_{k,g}\}| / N. \quad (17)$$

Notably, although the reward requires ground-truth information (e.g., the optimal objective value) from the MTOP instances, it is only used during the learning stage. In the inference stage, the reward signal is not required for the learned agent. If the optimal objective value is not accessible during the learning stage, we assume that a set of evaluated solutions $\mathbb{D}_k = \{(x_i, f_k(x_i))\}_{i=1}^{|\mathbb{D}_k|}$ on the training task instances is always available, from which the optimal objective value can be approximated as $\hat{f}_k^* = \mathbb{E}_{f \sim p(f|\mathbb{D}_k)} [\min f(x)]$.

5) *Agent and Environment*: We use the multilayer perceptron, a feed-forward neural network denoted as $\phi(s; \theta)$ to parameterize the agent $\pi(a|s; \theta)$, given the powerful expressiveness and function fitting capability of neural networks. That is, θ is the parameters of a neural network including the weights and biases of different layers. The input of the network is the state vector by concatenating the common features and task-specific features of each task with dimensionality equal to $4 + 7 \cdot K$. The output of the network is the action vector containing each task's individual KT action with three dimensions, leading to a total dimension of $3 \cdot K$. Moreover, to handle the continuous action space, we employ a Gaussian sampling to obtain the action as

$$\pi(a|s, \theta) = \frac{1}{\sigma \sqrt{2\pi}} \exp\left(-\frac{(a - \phi(s; \theta))^2}{2\sigma^2}\right) \quad (18)$$

The EMT environment refers to the MTOP instances with which the agent is interacting. Specifically, given an agent $\pi(a|s; \theta)$, the state transition probability $p_{tr}(s_{t+1}|s_t, a_t)$ is mainly affected by the distribution of MTOP instances since the behaviors of EMT algorithms are driven by the returned fitness of the handling tasks. To implement the environment,

we presume the existence of a task instance set, denoted as Θ , where each instance is an optimization task with a known optimal objective value and is assumed to be independently and identically (i.i.d.) drawn from a task distribution $p(f)$, where $f : x \rightarrow y$ denotes a task to be optimized. We also assume that the distribution of MTOP instances solely relies on the task distribution $p(\mathcal{T}) = p(f_1, \dots, f_K) = \prod_{k=1}^K p(f_k)$. Therefore, we can construct an MTOP instance by randomly sampling K tasks from the task instance set Θ . For more details on the MTOP instance construction, refer to Section S.III in the supplementary material [25].

C. Learning and Utilizing the Learnable Agent

1) *Learning Stage*: Having the definition of the agent and the environment, we should now determine how to efficiently learn the agent. To this end, we adopt the PPO which is good at handling continuous action space for learning the policy. In specific, the PPO algorithm contains an actor network $\pi(a|s; \theta)$ parameterized by θ to take actions and a critic network $\psi(s; \omega)$ parameterized by ω to estimate the value of the state. In our implementation, the critic network shares the same architecture as the actor network but with different parameters. The objective function of the actor network is

$$L_{\pi}(\theta) = \mathbb{E}_t[\min(\rho_t(\theta)\hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)] \quad (19)$$

where $\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma \lambda)^l (r_{t+l} + \psi(s_{t+l+1}; \omega') - \psi(s_{t+l}; \omega'))$ is the generalized advantage estimation (GAE) function, γ is the discounted factor, λ is a GAE parameter, and $\rho_t(\theta) = \pi(a_t|s_t; \theta) / \pi(a_t|s_t; \theta')$ is the probability ratio. Let θ', ω' be the parameters of actor and critic networks that are used in collecting the latest rollout data buffer. The objective function of the critic network is

$$L_{\psi}(\omega) = -\sum_{t=1}^T (\hat{A}_t + \psi(s_t; \omega') - \psi(s_t; \omega))^2. \quad (20)$$

Then the actor and critic networks are updated by gradient ascent. The rollout process conducted by the agent with base solver DE is given in Algorithm 1. A rollout process is terminated until the maximum rollout generations, denoted as G_{roll} is reached. In our implementation, we only consider MTOP instances with two tasks ($K = 2$). That is, the input MTOP instance is a task pair. Notably, our framework can be easily extended to solve MTOPs with more than two tasks by randomly pairing the tasks into multiple sub-MTOPs and using the learned agent along with a DE solver to solve them. The main functionality of the rollout is to collect experiences for learning by interacting with the environment.

To reduce environment variance for boosting learning efficiency, our implementation employs two tricks, i.e., pseudo-random initialization and deterministic KT intensity. For initialization, we bypass the usual uniform sampling at each rollout's start, opting instead to seed each task with a randomly chosen population from a pregenerated set (line 1). This primes the learning environment with a consistent baseline. With regards to KT intensity, we move away from probabilistic decisions for each offspring. Instead, we calculate the precise quota of KT solutions using $N_{k,\text{KT}} = \lceil 0.5 \cdot a_{k,1} \rceil$. This allows

Algorithm 1: Agent Rollout With Base Solver DE

Input: Task pair $\mathcal{T}=\{f_1, f_2\}$, base solver $\mathcal{A} = DE$, maximum rollout generations G_{roll} , parameterized agent $\pi(a|s, \theta)$, initial population set $\mathcal{P} = \{P_1, \dots, P_{N_P}\}$, population size per task N

Output: Rollout data buffer $\mathcal{D}$

```

1  $\mathcal{D} = \emptyset$ ; // empty rollout data buffer
2 Initialize population  $X$  of size  $N$  for each task by randomly selecting
  initial population from  $\mathcal{P}$ ;
3 Evaluate fitness of  $X$  to obtain fitness  $Y$  for each task;
4 Calculate initial state  $s$  by concatenating  $O_c$  and  $O_t$ ;
5  $g = 0$ ;
6 while  $g < G_{\text{roll}}$  do
7    $a \sim \pi(a|s, \theta)$ ; // sample action
8   foreach task  $f_k$  do
9     Retrieve KT action parameters  $a_{k,1}, a_{k,2}, a_{k,3}$  for task  $f_k$  from
       $a$ ;
10    Sample offspring population  $U$  by (1)-(2);
11     $N_{k,\text{KT}} = \lceil 0.5 \cdot a_{k,1} \rceil$ ; // compute KT quota
12    Randomly select  $N_{k,\text{KT}}$  indices from  $\{1, \dots, N\}$  to construct an
      index set  $\mathcal{I}_{\text{KT}} = \{j_1, \dots, j_{N_{k,\text{KT}}}\}$ ;
13    foreach index  $j$  in  $\mathcal{I}_{\text{KT}}$  do
14      Sample  $v_{k,j}$  by (9)-(10) with KT action parameters
         $a_{k,2}, a_{k,3}$ ;
15      Perform crossover to obtain  $u_{k,j}$  by (2);
16      Replace  $j$ th solution in  $U$  with  $u_{k,j}$ ;
17    end
18    Evaluate fitness  $Y_k$  of  $U$  on  $f_k$ ;
19    Calculate task-specific features  $O_{t,k}$  of task  $f_k$ ;
20    Calculate reward  $r_k$  based on (13);
21    Update populations  $X_k$  by selection;
22  end
23   $r = r_1 + r_2$ ; // sum up rewards of all tasks
24  Calculate common features  $O_c$ ;
25  Update state  $s$  by concatenating  $O_c$  and  $O_t$ ;
26   $\mathcal{D} = \mathcal{D} \cup (s, a, r)$ ;
27   $g = g + 1$ ;
28 end

```

us to construct the offspring population in two clear steps: first, by generating solutions via the DE solver, and second, by substituting $N_{k,\text{KT}}$ of these with KT-generated counterparts. This method ensures a balanced integration of KT without reliance on chance, streamlining the learning process.

The learning process of the agent with PPO is given in Algorithm 2. Specifically, to improve the sample efficiency, we adopt multiple parallelized environments for agents to simultaneously collect rollout data. The initial population set is constructed by conducting Latin hypercube sampling independently for N_P times, where N_P denotes the initial population set size. Moreover, in order to demonstrate the generality and compatibility of the proposed L2T framework under different base solvers, we choose the genetic algorithm (GA) as an additional example and incorporate the L2T with GA. GA involves a solution pairing process by the selection operator followed by the pairwise solution crossover, whose workflow is quite different from DE. Therefore, we implement the rollout process based on GA in a style similar to MFEA. The detailed rollout process with GA as the base solver is shown in Algorithm S.1 in Section S.II of the supplementary material [25].

2) *Inference Stage*: Upon completion of the agent's learning, it becomes ready for seamless integration with the DE base solver to tackle new MTOPs. The EMT algorithm we have developed, termed Multitask DE based on L2T

Algorithm 2: Agent Learning by PPO

Input: Task instance set $\Theta = \{f_1, \dots, f_K\}$, base solver $\mathcal{A}$, maximum rollout generations G_{roll} , maximum time steps T , number of parallel environments N_{env} , initial population set size N_P , rollout data buffer size for PPO update N_{buff}

Output: Learned agent $\pi(a|s, \theta^*)$

```

1 Pregenerate a set of initial populations  $\mathcal{P} = \{P_1, \dots, P_{N_P}\}$  by
  performing Latin hypercube sampling  $N_P$  times;
2 Initialize the actor network with parameter  $\theta$  and critic network with
  parameter  $\omega$ ;
3  $t = 0$ ;
4 while  $t < T$  do
5    $\mathcal{D} = \emptyset$ ;
6   while  $|\mathcal{D}| < N_{\text{buff}}$  do
7     Sample from  $\Theta$  to get MTOP instances  $\{\mathcal{T}_i\}_{i=1}^{N_{\text{env}}}$ ;
      // Algorithm 1
8     Send MTOP instances to remote workers to perform rollout
      by the agent with parameter  $\theta$ ;
9     Retrieve data buffer from worker environments and merge
       $\mathcal{D}_m = \bigcup_{i=1}^{N_{\text{env}}} \mathcal{D}_i$ ;
10     $\mathcal{D} = \mathcal{D} \cup \mathcal{D}_m$ ;
11     $t = t + N_{\text{env}} \cdot G_{\text{roll}}$ ;
12  end
13  Update actor network parameter  $\theta = \theta + \eta \nabla_{\theta} L_{\pi}$ ;
14  Update critic network parameter  $\omega = \omega + \eta \nabla_{\omega} L_{\psi}$ ;
15  Record the best-found agent parameter  $\theta^*$ ;
16 end

```

(MTDE-L2T), is detailed in Algorithm S.2 of the supplementary material. MTDE-L2T mirrors the structure of the rollout process in Algorithm 1, with the notable exception of delivering optimized solutions for individual tasks. It also omits the optimization data collection and reward computation steps found in the rollout process. Importantly, MTDE-L2T accommodates varying computational resources by allowing a flexible maximum number of generations G_{max} as input during testing, which can differ from the learning stage's G_{roll} . In cases where the GA serves as the base solver, we detail the Multitask GA based on L2T (MTGA-L2T) in Algorithm S.3 in Section S.II of the supplementary material [25].

D. Time Complexity Analysis

The time complexity of the proposed MTDE-L2T and MTGA-L2T when solving new MTOPs in the inference stage is analyzed in this subsection. Since the base solver is a customizable setting for users, we leave it out and mainly discuss the worst-case complexity of the L2T-related operations, including state feature calculation, action prediction, and offspring generation by KT. We treat the operation performed on a single value such as a dimension of or a fitness of the solution as the basic operation to derive the complexity. We denote the number of tasks, dimensionality, and population size as K, D, N , respectively. By summing up the common features and task-specific features, the feature computation cost is $\mathcal{O}(1 + D + (2N + 1)D + (2N + 1)D + K \cdot (1 + 1 + N^2 + (ND + D) + 1 + 1 + 1)) = \mathcal{O}(KN^2 + KND)$. The cost of action prediction is a function of the neural network parameters, denoted as $W(4 + 7K, 3K)$, which correlates with the input feature dimension of $4 + 7K$ and the output action dimension of $3K$. The number of network parameters can be considered as a linear function relating to input and output size dominated by K . Hence, we have the prediction cost $\mathcal{O}(K)$.

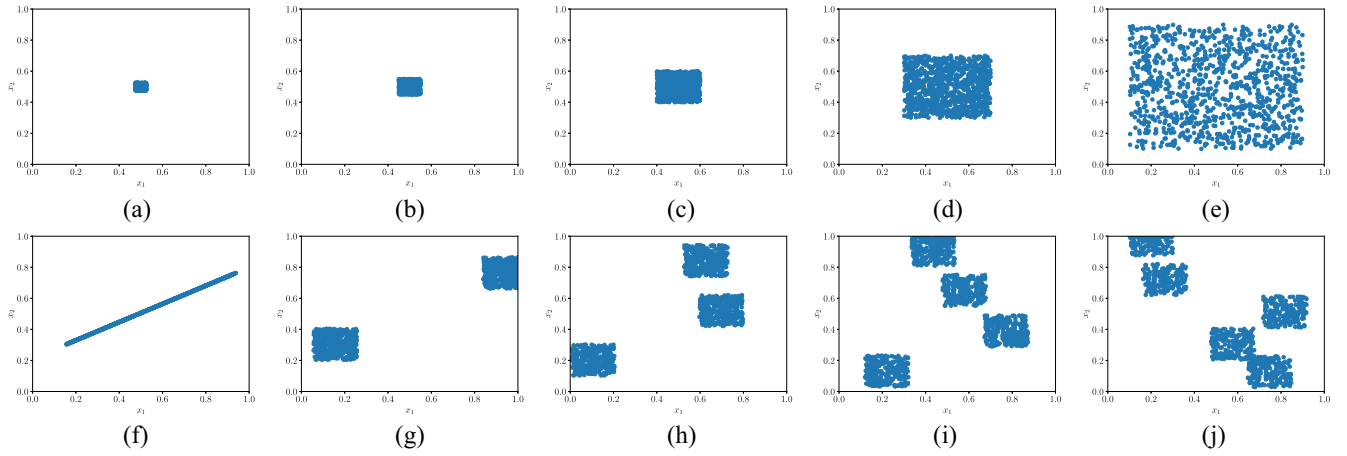


Fig. 2. First two dimensions of different global optimum distributions for defining tasks, which varies from the range of optimal solutions (a)–(e) and the number of distributed clusters (f)–(j). (a) Very small range (VS). (b) Small range (S). (c) Medium range (M). (d) Large range (L). (e) Very large range (VL). (f) Single cluster (C1). (g) Two clusters (C2). (h) Three clusters (C3). (i) Four clusters (C4). (j) Five clusters (C5).

The cost of offspring generation by KT is $\mathcal{O}(KND)$. Finally, by putting them all together and simplifying the nondominant terms, we obtain the complexity of L2T-related operations as $\mathcal{O}(KN^2 + KND)$.

IV. EXPERIMENTAL STUDY

A. Experimental Setup

1) *Problem Setting and Parameter Configuration*: To examine the adaptability of the EMT algorithms, we proposed two test suites with one constructed based on the CEC17MTOP benchmark [40] and the other one built upon the black-box optimization benchmark (BBOB) [41]. For the first test suite, we use basic functions with configurable optimum from CEC17MTOP benchmark [40] and define ten sets with different distributions of the global optimum for defining optimization tasks. The defined MTOP sets have various properties on global optimum distribution range and the number of distributed clusters in the solution space. The distribution of the first two dimensions of the task optimum of the ten sets is shown in Fig. 2. For the second test suite, we use 24 synthetic functions from BBOB with diverse fitness landscapes to serve as the component task of the MTOP instance. Hence, the second test suite poses greater challenges to the adaptability of EMT algorithms. We constructed 15 MTOP sets for testing the adaptability and one MTOP set for learning the agent. For the details of the problem setting and parameter configuration, refer to Section S.III in the supplementary material [25].

2) *Compared Algorithms*: Recall the motivation is to improve the adaptability of the implicit EMT, our L2T framework should be compared with other implicit EMT algorithms to demonstrate its effectiveness. Moreover, to the best of our knowledge, up to now, there exist no algorithms also working on automatically adapting the KT process allowed for comparison. We adopt two types of EMT algorithms with DE and GA as base solvers which will be compared with MTDE-L2T and MTGA-L2T, respectively. By keeping the base solver the same for comparison, the experimental results can fairly reflect which KT design contributes to more search efficiency gain. Regarding the

DE-based implicit EMT algorithms, the compared algorithms include AEMTO [26], MFDE [27], MKTDE [21], MTDE-AD [29], MTDE-B [28]. Regarding the GA-based implicit EMT algorithms, the compared algorithms include MFEA [3], MFEA2 [23], Generalized MFEA (GMFEA) [42], MFEA with Adaptive KT (MFEA-AKT) [18], and MTEA-AD [29]. The reason for choosing these algorithms for comparison is that they share similar structures with the KT action design in the L2T framework. For example, MFDE transfers differential vectors, and MTDE-B transfers base vectors. In fact, MFDE is a special case of the agent with fixed output action $a_1 = 0.5, a_2 = 0, a_3 = 1$. Therefore, comparing these EMT algorithms can reveal whether our L2T framework is capable of discovering a better KT process than the human-designed KT process.

Performance Metric: Each MTOP instance set contains 100 MTOP instances constructed either by sampling the task optimum from the predefined distribution specified in Table S.II of the supplementary material or by randomly selecting pair of function ID $fid \in \mathcal{F}$ and seed ID $sid \in \mathcal{S}$, respectively, as detailed in Table S.III in the supplementary material [25]. For each MTOP instance, we conduct 20 independent runs of each EMT algorithm, tracking fitness across generations. We conduct the Wilcoxon rank sum test with $\alpha = 0.05$ to statistically compare the L2T-based EMT algorithm against baselines for each task, treating all tasks as equally important. Unlike prior work, we devise a more strict pairwise comparison criterion as follows.

Definition 1: Given an MTOP instance $\mathcal{T} = \{f_k\}_{k=1}^K$, a target algorithm $\mathcal{A}_t$ and a baseline algorithm $\mathcal{A}_b$, with the set $\mathcal{V}_{\mathcal{A},k} = \{f_k(x_{k,g,r}^*)\}_{r=1}^R$ denoting the best-found fitness at generation g of task f_k over R independent runs on $\mathcal{T}$ by $\mathcal{A}$, $\mathcal{A}_t$ is said to outperform $\mathcal{A}_b$, denoted as $\mathcal{A}_t < \mathcal{A}_b$, if and only if $\mathcal{V}_{\mathcal{A}_t,k} \leq \mathcal{V}_{\mathcal{A}_b,k} \quad \forall k \in 1, \dots, K$, where the $\leq$ denotes the hypothesis test for examining whether the sample $\mathcal{V}_{\mathcal{A}_t,k}$ is significantly better than or the same as $\mathcal{V}_{\mathcal{A}_b,k}$. If the hypothesis test results are statistically same on all tasks, the comparison result between two algorithms is marked as a tie, denoted as $\mathcal{A}_t \approx \mathcal{A}_b$. Otherwise, the EMT algorithm $\mathcal{A}_t$ is considered to be worse than $\mathcal{A}_b$, $\mathcal{A}_t > \mathcal{A}_b$.

This criterion implies that sacrificing the optimization performance on one task to achieve performance improvement on the other task is not encouraged for an EMT algorithm. That is, if A_t outperforms A_b on some tasks but worse on others, it is marked as a loss. We report the comparative outcome of the L2T-based and peer EMT algorithms over all instances as the “Win/Tie/Lose” (W/T/L) count.

B. Effectiveness of the L2T Framework

One desired property of the proposed L2T framework is that the policy for performing KT can be efficiently and flexibly learned and utilized for the problem distribution of interest. To this end, we conduct experiments to perform policy learning with DE as the base solver on ten problem sets, respectively, based on CEC17MTOPT and two problem sets based on BBOB. These 12 MTOPT instance sets are with different distributions. Note that in this experiment the training and testing MTOPT instances are drawn from the same distribution. Therefore, learning on diverse MTOPT distributions can examine the adaptability of L2T to deal with complex MTOPT instances with different levels of task similarities. The comparative results between MTDE-L2T and other implicit EMT algorithms are shown in Table S.V of the supplementary material [25]. The results show that our proposed MTDE-L2T outcompetes peer implicit EMT algorithms on most of the MTOPT sets with varying task optimum distributions. This indicates the effectiveness of the proposed L2T framework in finding and adapting a proper KT process for solving unseen MTOPTs with complex distribution of interest.

Moreover, we investigate whether the agent learned by the L2T framework can bring more KT benefits with respect to its single-task counterpart, compared with other implicit EMT algorithms. To this end, we select implicit EMT algorithms using the same base solver DE on 10 MTOPT sets of the first test suite and calculate the *positive transfer rate* as

$$\sum_{i=1}^{N_{\mathcal{T}}} \mathbb{I}_{\mathcal{T}_i}(\mathcal{A}_{\text{EMT}} < \mathcal{A}_{\text{single}}) / N_{\mathcal{T}} \quad (21)$$

where $\mathcal{T}_i$ denotes the i th MTOPT instance, $\mathbb{I}(\cdot)$ denotes the indicator function, $N_{\mathcal{T}} = 100$ denotes the number of MTOPT instances drawn from a problem set, $\mathcal{A}_{\text{EMT}}$ denotes a candidate EMT algorithm and $\mathcal{A}_{\text{single}}$ denotes the single-task DE (STDE) without performing any KT between tasks for solving the MTOPT, and $<$ is the comparison criterion given in Definition 1. The results reported in Fig. 3 show that for most implicit EMT algorithms, the ability to offer positive transfer results tends to deteriorate as the range of MTOPT distribution becomes broader [Fig. 3(a)] and the modality of the distribution becomes more complex [Fig. 3(b)], while our proposed L2T-based EMT algorithm still yields high positive transfer rates.

C. Adaptability of the Learned Agent

We pretrained two agents with DE and GA as base solvers, respectively, on the BBOB_{learn} and then examine the adaptability of MTDE-L2T and MTGA-L2T equipped with the learned agents on diverse MTOPT sets based on BBOB (i.e., BBOB1-BBOB15). The tested MTOPT sets involve function classes with distinct characteristics and o.o.d. MTOPT instances

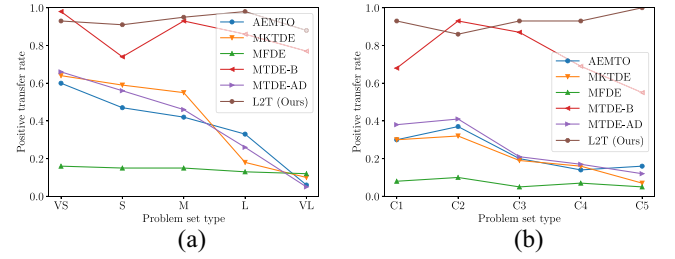


Fig. 3. Positive transfer rates of implicit EMT algorithms on problem sets with (a) varied task optimum range and (b) varied number of clusters.

that is unseen during training. Hence, these scenarios are considered more challenging. Moreover, as we aim to obtain an adaptive and versatile agent as well as possible, we invest more computational budget with time steps $T = 5e6$.

1) *Comparison With Implicit EMT Algorithms:* The comparative results of the proposed MTDE-L2T and MTGA-L2T and peer implicit EMT algorithms at generation= G_{roll} are shown in Table III. Specifically, we obtain three observations.

First, the MTDE-L2T and MTGA-L2T generally outperform their competitors on MTOPT sets (BBOB1-BBOB8) following nearly i.i.d. distribution with the training MTOPT set BBOB_{learn}. These results indicate that the learned agent can successfully adapt to unseen and i.i.d MTOPT instances to achieve competitive results. Second, the agent obtained by L2T can efficiently solve the o.o.d MTOPT instance set (BBOB9-BBOB15) containing heterogeneous task functions that have not been seen during the learning stage. These results indicate that the learned agent has the desired capability as we expect, i.e., adapting to a broad range of heterogeneous MTOPT instances. Third, for the MTOPT set using a single type of function but with different global optimum as tasks (e.g., BBOB3-BBOB8), the L2T-based algorithms also exhibit performance superiority over the compared EMT algorithm. This indicates that the learned agent also learns the effective skill of performing KT between tasks having homogeneous or highly similar function landscapes but with rather different optimums. As discussed in [12], the ability to utilize the shift invariance between tasks to speed up the search is also a desired property of KT. Moreover, it should be highlighted that MTDE-L2T with the learned agent generally outperforms the manually designed MFDE and MTDE-B whose KT behaviors actually lie in our designed action space. These results demonstrate the promise of our L2T framework in automatically discovering efficient KT processes for handling MTOPTs with broader distributions than the traditionally designed KT process driven by human expertise.

Next, we report the comparative results between L2T-based and other implicit EMT algorithms at generation= G_{max} in Table S.VI in the supplementary material [25]. Note that since the optimization steps that can be experienced by the agent during the learning stage is $G_{\text{roll}} = 100 < G_{\text{max}}$, the performance results at generation= G_{max} actually reflect the adaptability of the agents to generalize to longer optimization horizon (i.e., the evolution stage that has not seen before). From the table, we find that the performance advantage of L2T-based algorithms at generation= G_{max} has

a drop compared with that at generation= G_{roll} . However, the L2T-based EMT algorithms still offer performance advantages over their competitors, demonstrating the capability of the learned agent to provide long-term efficiency benefits in EMT. Moreover, we have conducted comparison with the recently proposed RL-based EMT method, i.e., MFEA with RL (MFEA-RL) [36]. The comparative results are given in Table S.VII of the supplementary material [25]. We observe that the proposed EMT algorithm equipped with pretrained agent generally outperforms MFEA-RL.

2) *Comparison With Explicit EMT Algorithms*: Besides the comparison with implicit EMT algorithms, we investigate how well the L2T-based algorithms perform when compared with state-of-the-art explicit EMT algorithms that involve an explicit learning process. We choose MTDE with explicit autoencoding (MTDE-EA) [13] and affine transformation-based MFEA (ATMFEA) [43] as the DE-based and GA-based explicit EMT algorithms, respectively, for comparison. The comparative results at different generations G_{roll} , G_{max} are reported in Table S.VIII in the supplementary material [25]. The results show that the performances of the proposed L2T-based algorithms are generally better than the compared explicit EMT algorithms in terms of the adaptability to various MTOPs with different distributions, further illustrating the preeminence of the L2T framework.

D. Transferability of the Learned Agent

To assess the reusability and transferability of the agent pretrained on $\text{BBOB}_{\text{learn}}$ to solve new MTOP sets, we conduct experiments on 12 problem sets (ten sets based on CEC19MTOP plus BBOB9 and BBOB10) by fine-tuning the agent that is pretrained on $\text{BBOB}_{\text{learn}}$. The agents in Section IV-B are regarded as learning from scratch since it does not reuse well-trained agents from other problem sets. The resultant algorithms with the agent learned from scratch and learned by fine-tuning are termed MTDE-L2T without fine-tuning (MTDE-L2T-w/o-FT) and MTDE-L2T by fine-tuning (MTDE-L2T-FT), respectively. The comparative results between MTDE-L2T-w/o-FT and MTDE-L2T-FT and DE-based EMT algorithms are shown in Table S.IX within the supplementary material [25]. We gain two observations from Table S.IX in the supplementary material. First, the learned MTDE-L2T generally outperforms the peer EMT algorithms, indicating the applicability of the proposed L2T framework on different problem sets of interest. Second, MTDE-L2T-FT yields better results than the MTDE-L2T-w/o-FT, illustrating the transferability of the pretrained agent on $\text{BBOB}_{\text{learn}}$ to new problem sets. Furthermore, the performance along the learning process of fine-tuning (L2T-FT) and learning from scratch (L2T-w/o-FT) on five problem sets are shown in Fig. 4. It can be seen that on most problem sets, L2T-FT converges faster and better than L2T-w/o-FT, indicating that L2T-FT with the pretrained agent normally requires less computational effort to achieve high performance. These results further demonstrate the transferability of the proposed L2T framework.

E. Component Analysis

We first analyze the effectiveness of the learning process of the L2T framework. To this end, several agents with fixed action and random action are formulated. Specifically, we use “MTDE-f(x,y,z)” to denote the EMT algorithm with an agent that simply uses a fixed action $a_1 = x$, $a_2 = y$, $a_3 = z$ for each task along the search process. Moreover, we use “MTDE-r” to denote the agent that randomly draws a sample from $[0, 1]^3$ as the action to perform KT for each task each time. Furthermore, the learned agent should be compared with the STDE that does not transfer any cross-task information, to reveal that the performance gain is actually caused by the effective utilization of transferable knowledge between tasks. The comparative results between the MTDE-L2T and the above-mentioned algorithms are given in Table S.X of the supplementary material. The results show that MTDE-L2T yields significantly better results than the compared algorithms. These results indicate that: 1) the learning algorithm PPO used in the L2T framework helps find well-performing agents than the trivial agents with fixed and random actions and 2) the L2T framework successfully learns to identify and harness useful cross-task knowledge to improve search efficiency.

Next, we conduct ablation studies to verify the designed components in the L2T framework indeed contribute to the outcomes of good performance. In particular, the plausibility and the effectiveness of the action space design, extracted features as state space, and the reward design with two proposed additional reward terms should be examined. Regarding the effects of action space, we, respectively, remove each action from the designed 3-D action space to learn the agent. Then, we formulate three variants, namely, “L2T-w/o- a_1 ,” “L2T-w/o- a_2 ,” and “L2T-w/o- a_3 .” Accordingly, the number of output units of the actor network is reduced to $2 \cdot K$, while the default settings of the removed actions are set as $a_1 = 0.5$, $a_2 = 0$, $a_3 = 0$, respectively. Regarding the effects of the observation space, we, respectively, remove the common features O_c and task-specific features O_t from the original designed state space, to formulate two variants, i.e., L2T-w/o- O_c and L2T-w/o- O_t . The modification on the actor networks of L2T-w/o- O_c and L2T-w/o- O_t is similar to that of L2T-w/o- a_1 except that the number of input units will be reduced accordingly. To verify the overall effects of the devised feature extraction, we formulate a variant named “L2T-w/o-FE” that uses the full state of the population and fitness of all the tasks discussed in Section III-B3). Regarding the effects of the reward design, we formulate two variants called “L2T-w/o- r_{conv} ” and “L2T-w/o- r_{KT} ,” where the reward term r_{conv} and r_{KT} are removed from them, respectively. Note that each of the formulated variants is independently learned on a training problem sets and subsequently evaluated on a testing problem set sharing the same distribution with the training set, consistent with the approach taken by MTDE-L2T.

The results between the MTDE-L2T and the above-mentioned variants are reported in Table S.XI in the supplementary material [25]. It can be observed that the proposed MTDE-L2T obtains more “+” than “−” when compared with L2T-w/o- a_1 . This result indicates that the transfer

TABLE III
COMPARATIVE RESULTS BETWEEN THE PROPOSED L2T-BASED AND OTHER IMPLICIT EMT ALGORITHMS AT GENERATION= G_{ROLL}

Problem	MTDE-L2T vs					MTGA-L2T vs				
	AEMTO	MFDE	MKTDE	MTDE-AD	MTDE-B	GMFEA	MFEA	MFEA-AKT	MFEA2	MTEA-AD
BBOB1	56/43/1(+)	63/30/7(+)	88/10/2(+)	53/46/1(+)	36/59/5(+)	83/10/7(+)	82/13/5(+)	84/12/4(+)	76/19/5(+)	76/12/12(+)
BBOB2	68/30/2(+)	61/32/7(+)	85/11/4(+)	57/38/5(+)	43/50/7(+)	84/9/1(+)	85/11/4(+)	77/18/5(+)	78/19/3(+)	78/10/12(+)
BBOB3	100/0/0(+)	14/84/2(+)	5/86/9(-)	100/0/0(+)	11/86/3(+)	100/0/0(+)	100/0/0(+)	100/0/0(+)	100/0/0(+)	100/0/0(+)
BBOB4	30/69/1(+)	12/84/4(+)	98/2/0(+)	7/91/2(+)	1/96/3(-)	100/0/0(+)	100/0/0(+)	100/0/0(+)	95/5/0(+)	100/0/0(+)
BBOB5	8/89/3(+)	5/88/7(-)	4/87/9(-)	7/93/0(+)	3/91/6(-)	91/7/2(+)	89/9/2(+)	86/11/3(+)	92/7/1(+)	86/11/3(+)
BBOB6	25/74/1(+)	40/59/1(+)	77/23/0(+)	12/83/5(+)	3/85/12(-)	12/71/17(-)	8/70/22(-)	10/71/19(-)	24/71/5(+)	7/56/37(-)
BBOB7	9/86/5(+)	8/89/3(+)	7/91/2(+)	6/90/4(+)	5/84/11(-)	18/77/5(+)	20/75/5(+)	20/77/3(+)	24/74/2(+)	1/79/20(-)
BBOB8	34/65/1(+)	23/74/3(+)	87/13/0(+)	13/85/2(+)	5/88/7(-)	98/2/0(+)	98/2/0(+)	88/12/0(+)	83/17/0(+)	100/0/0(+)
BBOB9	42/57/1(+)	51/35/14(+)	67/23/10(+)	45/52/3(+)	18/66/16(+)	59/33/8(+)	60/29/11(+)	60/28/12(+)	64/29/7(+)	63/21/16(+)
BBOB10	51/33/16(+)	55/36/9(+)	62/29/9(+)	54/35/11(+)	27/48/25(+)	78/16/6(+)	78/18/4(+)	79/18/3(+)	79/18/3(+)	72/21/7(+)
BBOB11	96/0/4(+)	27/70/3(+)	13/85/2(+)	96/0/4(+)	16/82/2(+)	100/0/0(+)	100/0/0(+)	100/0/0(+)	100/0/0(+)	99/0/1(+)
BBOB12	49/51/0(+)	1/30/69(-)	1/30/69(-)	45/53/2(+)	15/80/5(+)	76/22/2(+)	75/24/1(+)	83/16/1(+)	82/18/0(+)	86/13/1(+)
BBOB13	88/9/3(+)	28/68/4(+)	12/78/10(+)	94/5/1(+)	12/81/7(+)	99/0/1(+)	98/0/2(+)	99/0/1(+)	99/0/1(+)	98/0/2(+)
BBOB14	11/86/3(+)	10/87/3(+)	10/88/2(+)	6/89/5(+)	8/89/3(+)	26/74/0(+)	24/75/1(+)	30/70/0(+)	42/57/1(+)	0/81/19(-)
BBOB15	14/82/4(+)	15/79/6(+)	23/73/4(+)	18/78/4(+)	7/84/9(-)	25/51/24(+)	24/54/22(+)	20/56/24(-)	23/59/18(+)	23/54/23(=)

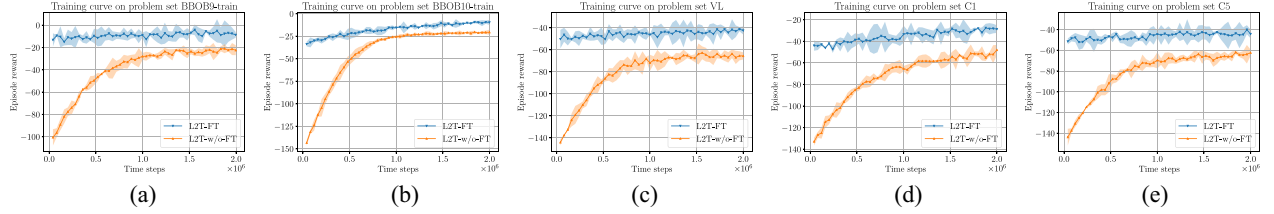


Fig. 4. Training performance of agents by fine-tuning (L2T-FT) and retraining from scratch (L2T-w/o-FT) on new problem sets with significantly different task distributions. (a) BBOB9. (b) BBOB10. (c) VL. (d) C1. (e) C5.

intensity needs to be adaptively adjusted according to the evolution status and our MTDE-L2T can automatically learn useful skills in deciding transfer intensity. When removing a_2 and a_3 from our design, we can also witness a degradation in the performance, which validates the effectiveness of the proposed action design in (10). Moreover, MTDE-L2T yields generally better results when comparing the variants with removed state features, i.e., L2T-w/o- O_c , L2T-w/o- O_t , and L2T-w/o- FE . This indicates that the proposed state feature extraction is beneficial by incorporating more intertask and intratask information to facilitate wise decision-making. Finally, MTDE-L2T's optimization performances surpass L2T-w/o- r_{conv} and L2T-w/o- r_{KT} , demonstrating the usefulness of the two proposed reward terms in assisting the agent to learn faster. One more notable observation is that the proposed reward term r_{KT} appears to contribute more to delivering high performance than r_{conv} .

F. Agent Behavior Analysis

The pattern of making KT decisions of the learned agent is analyzed in this section. The observed states and the actions taken by the agent along the search process on two MTO instances of the BBOB1 problem set are plotted in Fig. 5. The MTO15 and MTO23 are selected as representative cases to analyze the behaviors. MTO15 contains a unimodal function as task f_1 and a highly multimodal function as task f_2 with markedly different global optimum. MTO23 contains two unimodal functions as tasks with differences in the conditioning number of the landscape. It can be seen in Fig. 5 that when solving MTO15, the agent learns the behavior of performing unidirectional KT from task 1 to task 2 with ($a_1 \geq 0$) while keeping the task 2 evolve without KT ($a_1 \approx 0$). This learned strategy can be understood as using easy task

(task 1) to help difficult task (task 2) search efficiently, which has been reported to be useful in literature [42]. In the case of MTO15 having distant task optimum, the learned agent favors transferring differential vector rather than base vectors, i.e., $a_3 > a_2$ [Fig. 5(d)], the effectiveness of this KT behavior has also been validated by [21]. For MTO23 with two unimodal functions, the learned agent also manages to gradually adapt the KT action to improve search performance. First, we observe that the agent can dynamically adjust transfer intensity (a_1) and the way of performing KT (a_2, a_3) according to the evolution state [Fig. 5(g)]. Second, the agent learns to give a high base vector transfer ratio ($a_2 > 0$) in the early stage of the optimization process [Fig. 5(h)]. This behavior is reasonable since populations are highly overlapped in the beginning generations, due to the uniform initialization in the whole search space. Then base vector transfer between tasks in the early stage can help locate promising regions faster. As the search proceeds, the populations gradually converge to different optimum, the base vector transfer rate decreases [Fig. 5(h)] since direct solution transfer is not useful under this condition.

G. Parameter Sensitivity Analysis

We empirically find that the most sensitive parameters of the L2T are b_1, b_2 in the reward design. b_3 is related to the agent's episodic final performance and we find it appears to be insensitive. Hence, b_3 is suggested to be fixed. The component analysis in Section IV-E has suggested that setting $b_1 = 0$ or $b_2 = 0$ would lead to deteriorated results. The settings of b_1, b_2 reflect the balance between the convergence gain term r_{conv} and the KT gain term r_{KT} . Therefore, we mainly investigate the relative effects on the setting of b_1, b_2 by keeping $b_1 = 1$ unchanged and varying the parameter $b_2 \in$

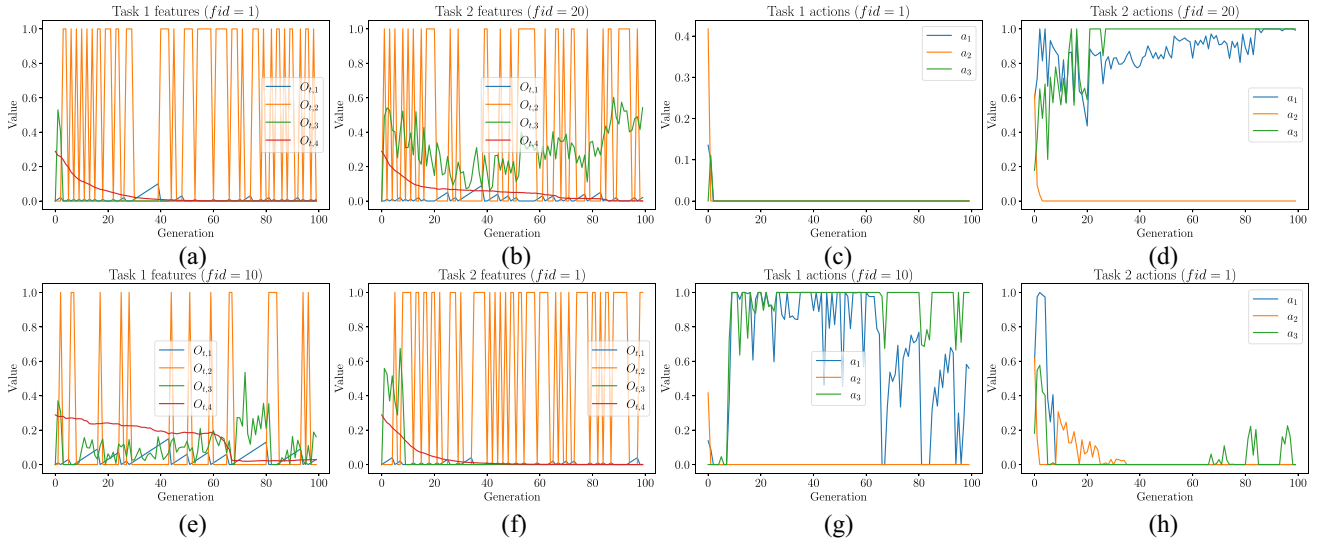


Fig. 5. Observations and the taken actions when solving two MTOP instances, MTOP15 (a)–(d) and MTOP23 (e)–(h) in the BBOB1 problem set.

{0.1, 0.5, 1, 5, 10, 50, 100}. Specifically, we learned the agent under different settings of b_2 and tested the performance of the M2DE-L2T on the 15 BBOB-based problem sets. We use the mean standard score (MSS) [40] as the metric to quantitatively measure the performance of different settings. Then the average ranking computed based on the MSS will be reported. The investigation results on parameter b_2 at generation= G_{roll} and generation= G_{max} are given in Tables S.XII and S.XIII in the supplementary material [25]. From the two tables, we can observe that for the BBOB problem set $b_2 = 10$ and $b_2 = 5$ yield the best performance for the optimization horizon of G_{roll} and G_{max} , respectively. When adopting the L2T framework to solve new MTOP sets, the grid search can be conducted on the parameter b_2 to fulfill its capability.

H. Further Analysis

1) *Time Efficiency*: To assess the time efficiency of the proposed L2T method, we have recorded the computational overhead, comprising the average time required to generate a single solution and update the population, in contrast to the time taken to evaluate a single solution. These metrics are depicted in Fig. S.1 in the supplementary material [25]. The black dashed line at $y = 0.1$ indicates that the computational overhead at this level corresponds to approximately 10% of the evaluation cost. Despite the integration of deep neural networks, the MTGA-L2T demonstrates a similar overhead to that of GA-based EMT algorithms. In contrast, the MTDE-L2T incurs the highest computational overhead among DE-based EMT algorithms. Notably, as the evaluation cost increases, the proportion of the computational overhead becomes negligible. Specifically, when the evaluation cost exceeds 0.001 s, the additional computational overhead attributed to the L2T method is reduced to no more than 10%. Considering the performance enhancements achieved by the L2T approach, we recommend its application in scenarios where the evaluation costs exceed 0.0001 s. In such scenario, the proposed EMT algorithms are expected to strike a good balance between the computational overhead and performance benefits.

2) *Effects of the RL Algorithms*: To investigate the effects of the RL algorithms for agent training, we conduct experiments on two advanced off-policy RL algorithm that can handle continuous actions, including soft actor-critic (SAC) [44] and twin-delayed deep deterministic policy gradient (TD3) [45], to compare with the PPO adopted in our method. The experimental results are given in Table S.XIV in the supplementary material [25]. The results show that in general, the agent trained by PPO outperforms the one trained with SAC while TD3 produces even better performance than PPO, suggesting that more advanced RL methods can further fulfill the potential of the proposed L2T framework.

3) *Comparison With Adaptive EC Algorithms*: The baseline single-task EC algorithms employed in previous experiments lack self-adaptation mechanisms. A critical distinction between the L2T and self-adaptive EC algorithms lies in the source of adaptive knowledge. While L2T adapts by leveraging KTred across tasks, self-adaptive EC algorithms rely solely on knowledge within the same task for adaptation. To further evaluate the effectiveness of L2T, we integrated it with the well-known adaptive DE algorithm, JADE [46]. For integration simplicity, JADE without an external archive is used as the base solver in L2T. The KT actions and the remaining components of L2T are maintained as originally designed. The agents of the multitask JADE with L2T (MTJADE-L2T) were trained on multiple MTOP sets independently. Then, they are tested on corresponding MTOP sets, containing unseen instances during training to assess adaptability. The experimental outcomes, as detailed in Table S.XV in the supplementary material [25], indicate that MTJADE-L2T generally surpasses single-task JADE (STJADE) across various MTOP sets. This performance superiority suggests that combining the self-adaptation mechanism, which utilizes intratask information, with L2T's approach to harnessing cross-task knowledge, yields significant benefits.

Notably, on problem sets where task optima are closely clustered (VS) or exhibit simpler patterns with few clusters (C1), the advantages of L2T are more pronounced. Conversely, in scenarios where task optima are markedly

different (VL) and patterns are more complex (C5), the performance disparities between MTJADE-L2T and STJADE are small. These observations imply that the efficacy of L2T depends on the strength of the base solver and the similarity of tasks, which have also been revealed in previous study [47]. For strong base solvers like STJADE, direct and highly similar knowledge from other tasks, as seen in conditions VS and C1, is particularly beneficial for further enhancements. Furthermore, complex scenarios such as VL and C5 may benefit from minimizing information transfer, essentially reverting to the STJADE. Luckily, MTJADE-L2T is found to be capable of learning this behavior from the distribution of MTOP instances, achieving performance comparable to STJADE.

I. Real-World Application

In this section, we investigate the performance of the proposed L2T framework in solving real-world application MTOPs. To this end, we employ the hyperparameter optimization (HPO) task for machine learning methods [48] as the testbed. We choose the HPO for testing the EMT algorithms for two reasons. First, this is a widely-used testbed for benchmarking black-box optimizers and the learning-to-optimize methods [19]. Second, despite working on different datasets, there have been some studies supporting that the hyperparameters can be transferred among different learning tasks to address the cold-start problem in the search [49]. In other words, jointly optimizing the hyperparameters of multiple learning tasks can be naturally formulated as an MTOP which is widely accepted by the machine learning community [50]. Motivated by this, we consider this new application problem to examine the capability of the EMT algorithms to perform effective KT.

Specifically, an HPO task can be specified by the tuple $(\mathcal{H}, \mathcal{D}, \mathcal{L})$ with a hyperparameter space $\mathcal{H}$, a dataset $\mathcal{D}$, and a learning algorithm $\mathcal{L}$. The goal of an HPO task is to find the best hyperparameter for the algorithm $\mathcal{L}$ to minimize the loss on the validation dataset. The hyperparameter space dimensionality is denoted as $D_{\mathcal{H}}$. Herein, given a learning algorithm $\mathcal{L}$, different datasets $\mathcal{D}$ leads to distinct instantiation of HPO task instances. For more details of the employed datasets in the benchmark, refer to [48]. Especially, we adopt three task sets for optimizing the hyperparameters of machine learning algorithms including support vector machine (SVM) with dimension $D_{\mathcal{H}} = 2$, XGBoost with dimension $D_{\mathcal{H}} = 8$, fully-connected neural networks (FCNets) with dimension $D_{\mathcal{H}} = 6$ on various datasets published in [51].

An MTOP instance is constructed by randomly sampling two tasks in the task set, the same way as the BBOB problem set does. Since the datasets in the benchmark are with diverse characteristics, randomly pairing the tasks to formulate the MTOP instance can testify the adaptability of the EMT algorithms to handle the diverse MTOPs containing both high-similarity tasks and low-similarity tasks. Then three HPO problem sets named SVM, XGBoost, and FCNet are formulated by constructing 100 MTOP instances based on each task set, respectively. We perform the policy learning

process from scratch on three problem sets and test on unseen MTOP instance set. The optimization horizon of learning and testing is set as $G_{\text{roll}} = G_{\text{max}} = 100$. To fulfill the potential of the L2T, we conduct a grid search on the parameter set $b_2 \in \{0.01, 0.1, 1, 10, 100\}$ to tune the parameter b_2 . The results turn out that $b_2 = 0.01$ works well for MTDE-L2T and $b_2 = 10$ works well for MTGA-L2T. Moreover, the target accuracy is set to $\xi_{\text{SVM}} = 1, \xi_{\text{XGBoost}} = 0.1, \xi_{\text{FCNet}} = 0.1$ for SVM, XGBoost, and FCNet, respectively. The results of MTDE-L2T and MTGA-L2T with the best parameter setting of b_2 compared with peer EMT algorithms on HPO problems are shown in Table S.XVI in the supplementary material [25]. It can be observed that the proposed L2T-based EMT algorithms offer competitive performance, demonstrating the effectiveness of the proposed L2T framework in solving real-world application MTOPs.

V. CONCLUSION

This study presents the L2T framework to refine the KT process's adaptability, which leverages RL to autonomously adapt KT intensity and strategies for the effective transfer of knowledge across tasks along the EMT process. This approach marks a departure from the conventional, expert-driven KT, significantly decreasing the need for specialized labor. By employing parameterized neural networks to model the agent for performing KT, the L2T framework empowers an automatic discovery of effective KT policies for the target MTOPs at hand. Our experimental study, encompassing both synthetic and real-world scenarios, confirms that the L2T framework not only improves the flexibility of the KT process but also broadens the implicit EMT algorithm's capacity to handle diverse and intricate MTOPs with increased efficiency.

Looking ahead, we recognize the challenge posed by the limited number of real-world problem instances and plan to explore instance generation [52] as a potential solution. Furthermore, it is worthy of exploring the representation learning approach to achieve automated state feature extraction besides manual feature extraction. Finally, we plan to extend the proposed L2T framework to solve more complex real-world MTOPs.

REFERENCES

- [1] Z. Li, Q. Zhang, X. Lin, and H.-L. Zhen, "Fast covariance matrix adaptation for large-scale black-box optimization," *IEEE Trans. Cybern.*, vol. 50, no. 5, pp. 2073–2083, May 2020.
- [2] Z. Guo, Y.-S. Ong, T. He, and H. Liu, "Co-learning Bayesian optimization," *IEEE Trans. Cybern.*, vol. 52, no. 9, pp. 9820–9833, Sep. 2022.
- [3] A. Gupta, Y.-S. Ong, and L. Feng, "Multifactorial evolution: Toward evolutionary multitasking," *IEEE Trans. Evol. Comput.*, vol. 20, no. 3, pp. 343–357, Jun. 2016.
- [4] K. C. Tan, L. Feng, and M. Jiang, "Evolutionary transfer optimization—A new frontier in evolutionary computation research," *IEEE Comput. Intell. Mag.*, vol. 16, no. 1, pp. 22–33, Feb. 2021.
- [5] S.-H. Wu, Z.-H. Zhan, K. C. Tan, and J. Zhang, "Orthogonal transfer for multitask optimization," *IEEE Trans. Evol. Comput.*, vol. 27, no. 1, pp. 185–200, Feb. 2023.
- [6] S. Tan, Y. Wang, G. Sun, T. Pang, and K. Tang, "A surrogate-assisted evolutionary framework for expensive multitask optimization problems," *IEEE Trans. Evol. Comput.*, early access, Feb. 28, 2024, doi: [10.1109/TEVC.2024.3370937](https://doi.org/10.1109/TEVC.2024.3370937).

- [7] Z.-F. Xue, Z.-J. Wang, Z.-H. Zhan, S. Kwong, and J. Zhang, "Neural network-based knowledge transfer for multitask optimization," *IEEE Trans. Cybern.*, vol. 54, no. 12, pp. 7541–7554, Dec. 2024.
- [8] E. O. Scott and K. A. De Jong, "First complexity results for evolutionary knowledge transfer," in *Proc. 17th ACM/SIGEVO Conf. Found. Genet. Algorithms*, 2023, pp. 140–151.
- [9] Z. Liu, G. Li, H. Zhang, Z. Liang, and Z. Zhu, "Multifactorial evolutionary algorithm based on diffusion gradient descent," *IEEE Trans. Cybern.*, vol. 54, no. 7, pp. 4267–4279, Jul. 2024.
- [10] Y. Tian, Y. Hu, H. Ma, L. Wu, and X. Zhang, "An evolutionary multitasking algorithm for efficient multiobjective recommendations," *IEEE Trans. Artif. Intell.*, vol. 6, no. 3, pp. 518–532, Mar. 2025.
- [11] Q.-T. Yang, X.-X. Xu, Z.-H. Zhan, J. Zhong, S. Kwong, and J. Zhang, "Evolutionary multitask optimization for multiform feature selection in classification," *IEEE Trans. Cybern.*, vol. 55, no. 4, pp. 1673–1686, Apr. 2025.
- [12] S.-H. Wu, Z.-H. Zhan, K. C. Tan, and J. Zhang, "Transferable adaptive differential evolution for many-task optimization," *IEEE Trans. Cybern.*, vol. 53, no. 11, pp. 7295–7308, Nov. 2023.
- [13] L. Feng et al., "Evolutionary multitasking via explicit autoencoding," *IEEE Trans. Cybern.*, vol. 49, no. 9, pp. 3457–3470, Sep. 2019.
- [14] K. Chen, B. Xue, M. Zhang, and F. Zhou, "An evolutionary multitasking-based feature selection method for high-dimensional classification," *IEEE Trans. Cybern.*, vol. 52, no. 7, pp. 7172–7186, Jul. 2022.
- [15] L. Feng et al., "Explicit evolutionary multitasking for combinatorial optimization: A case study on capacitated vehicle routing problem," *IEEE Trans. Cybern.*, vol. 51, no. 6, pp. 3143–3156, Jun. 2021.
- [16] Y. Chen, J. Zhong, L. Feng, and J. Zhang, "An adaptive archive-based evolutionary framework for many-task optimization," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 4, no. 3, pp. 369–384, Jun. 2020.
- [17] J. Zhang, W. Zhou, X. Chen, W. Yao, and L. Cao, "Multisource selective transfer framework in multiobjective optimization problems," *IEEE Trans. Evol. Comput.*, vol. 24, no. 3, pp. 424–438, Jun. 2020.
- [18] L. Zhou, L. Feng, K. C. Tan, J. Zhong, Z. Zhu, K. Liu, and C. Chen, "Toward adaptive knowledge transfer in multifactorial evolutionary computation," *IEEE Trans. Cybern.*, vol. 51, no. 5, pp. 2563–2576, May 2021.
- [19] Z. Ma et al., "Metabox: A benchmark platform for meta-black-box optimization with reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 10775–10795.
- [20] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [21] J.-Y. Li, Z.-H. Zhan, K. C. Tan, and J. Zhang, "A meta-knowledge transfer-based differential evolution for multitask optimization," *IEEE Trans. Evol. Comput.*, vol. 26, no. 4, pp. 719–734, Aug. 2022.
- [22] A. Gupta, Y.-S. Ong, and L. Feng, "Insights on transfer optimization: Because experience is the best teacher," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 2, no. 1, pp. 51–64, Feb. 2018.
- [23] K. K. Bali, Y.-S. Ong, A. Gupta, and P. S. Tan, "Multifactorial evolutionary algorithm with online transfer parameter estimation: MFEA-II," *IEEE Trans. Evol. Comput.*, vol. 24, no. 1, pp. 69–83, Feb. 2020.
- [24] W. Lin, Q. Lin, L. Feng, and K. C. Tan, "Ensemble of domain adaptation-based knowledge transfer for evolutionary multitasking," *IEEE Trans. Evol. Comput.*, vol. 28, no. 2, pp. 388–402, Apr. 2024.
- [25] S.-H. Wu, Y. Huang, X. Wu, L. Feng, Z.-H. Zhan, and K. C. Tan, "Supplementary material for 'learning to transfer for evolutionary multitasking.'" 2024. [Online]. Available: <https://wushenghao8404.github.io/files/L2T-supplement.pdf>
- [26] H. Xu, A. K. Qin, and S. Xia, "Evolutionary multitask optimization with adaptive knowledge transfer," *IEEE Trans. Evol. Comput.*, vol. 26, no. 2, pp. 290–303, May 2021.
- [27] L. Feng et al., "An empirical study of multifactorial PSO and multifactorial DE," in *Proc. IEEE Congr. Evol. Comput. (CEC)*, 2017, pp. 921–928.
- [28] C. Jin, P.-W. Tsai, and A. K. Qin, "A study on knowledge reuse strategies in multitasking differential evolution," in *Proc. IEEE Congr. Evol. Comput. (CEC)*, 2019, pp. 1564–1571.
- [29] C. Wang, J. Liu, K. Wu, and Z. Wu, "Solving multitask optimization problems with adaptive knowledge transfer via anomaly detection," *IEEE Trans. Evol. Comput.*, vol. 26, no. 2, pp. 304–318, Apr. 2022.
- [30] Y. Wang, B. Xin, B. Liu, and Q. Wang, "Automated design of collaboration-based hybrid metaheuristics," *IEEE Trans. Cybern.*, vol. 54, no. 12, pp. 7877–7890, Dec. 2024.
- [31] W. Yi, R. Qu, L. Jiao, and B. Niu, "Automated design of metaheuristics using reinforcement learning within a novel general search framework," *IEEE Trans. Evol. Comput.*, vol. 27, no. 4, pp. 1072–1084, Aug. 2023.
- [32] J. Sun, X. Liu, T. Bäck, and Z. Xu, "Learning adaptive differential evolution algorithm from optimization experiences by policy gradient," *IEEE Trans. Evol. Comput.*, vol. 25, no. 4, pp. 666–680, Aug. 2021.
- [33] Z.-H. Zhan, J.-Y. Li, S. Kwong, and J. Zhang, "Learning-aided evolution for optimization," *IEEE Trans. Evol. Comput.*, vol. 27, no. 6, pp. 1794–1808, Dec. 2023.
- [34] Y. Jiang, Z.-H. Zhan, K. C. Tan, and J. Zhang, "Knowledge learning for evolutionary computation," *IEEE Trans. Evol. Comput.*, vol. 29, no. 1, pp. 16–30, Feb. 2025.
- [35] H. Zhang, J. Sun, and Z. Xu, "Learning to mutate for differential evolution," in *Proc. IEEE Congr. Evol. Comput. (CEC)*, 2021, pp. 1–8.
- [36] S. Li, W. Gong, L. Wang, and Q. Gu, "Evolutionary multitasking via reinforcement learning," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 8, no. 1, pp. 762–775, Feb. 2024.
- [37] Z. Zhu, K. Lin, A. K. Jain, and J. Zhou, "Transfer learning in deep reinforcement learning: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 11, pp. 13344–13362, Nov. 2023.
- [38] Z. Liang, X. Xu, L. Liu, Y. Tu, and Z. Zhu, "Evolutionary many-task optimization based on multisource knowledge transfer," *IEEE Trans. Evol. Comput.*, vol. 26, no. 2, pp. 319–333, Apr. 2022.
- [39] S. Khadka and K. Tumer, "Evolution-guided policy gradient in reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 1196–1208.
- [40] B. Da et al., "Evolutionary multitasking for single-objective continuous optimization: Benchmark problems, performance metric, and baseline results," 2017, *arXiv:1706.03470*.
- [41] N. Hansen, A. Auger, R. Ros, O. Mersmann, T. Tušar, and D. Brockhoff, "COCO: A platform for comparing continuous optimizers in a black-box setting," *Optim. Methods Softw.*, vol. 36, no. 1, pp. 114–144, 2021.
- [42] J. Ding, C. Yang, Y. Jin, and T. Chai, "Generalized multitasking for evolutionary optimization of expensive problems," *IEEE Trans. Evol. Comput.*, vol. 23, no. 1, pp. 44–58, Feb. 2019.
- [43] X. Xue et al., "Affine transformation-enhanced multifactorial optimization for heterogeneous problems," *IEEE Trans. Cybern.*, vol. 52, no. 7, pp. 6217–6231, Jul. 2022.
- [44] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.
- [45] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1587–1596.
- [46] J. Zhang and A. C. Sanderson, "JADE: Adaptive differential evolution with optional external archive," *IEEE Trans. Evol. Comput.*, vol. 13, no. 5, pp. 945–958, Oct. 2009.
- [47] X. Xue, C. Yang, L. Feng, K. Zhang, L. Song, and K. C. Tan, "Solution transfer in evolutionary optimization: An empirical study on sequential transfer," *IEEE Trans. Evol. Comput.*, vol. 28, no. 6, pp. 1776–1793, Dec. 2024.
- [48] A. Klein, Z. Dai, F. Hutter, N. Lawrence, and J. Gonzalez, "Meta-surrogate benchmarking for hyperparameter optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 6267–6277.
- [49] P. Tighineanu, K. Skubch, P. Baireuther, A. Reiss, F. Berkenkamp, and J. Vinogradskaya, "Transfer learning with Gaussian processes for Bayesian optimization," in *Proc. Int. Conf. Artif. Intell. Stat.*, 2022, pp. 6152–6181.
- [50] K. Swersky, J. Snoek, and R. P. Adams, "Multi-task Bayesian optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 26, 2013, pp. 2004–2012.
- [51] J. Vanschoren, J. N. Van Rijn, B. Bischl, and L. Torgo, "Openml: networked science in machine learning," *ACM SIGKDD Explor. Newslett.*, vol. 15, no. 2, pp. 49–60, 2014.
- [52] K. Tang, S. Liu, P. Yang, and X. Yao, "Few-shots parallel algorithm portfolio construction via co-evolution," *IEEE Trans. Evol. Comput.*, vol. 25, no. 3, pp. 595–607, Jun. 2021.